BUILD / ITERATE / SHIP

Your first website with GitHub, Firebase, ChatGPT & Codex

A friendly, practical roadmap for going from a blank computer to a real website - without needing to become a professional developer first.

The basic loop
Describe one small change. Let Codex work in the project. Review it locally. Run checks. Commit it to GitHub. Publish a preview. Repeat.

WHAT YOU WILL HAVE AT THE END

[ ] A working website on your computer

[ ] A private or public GitHub repository with version history

[ ] A Firebase-hosted URL you can share

[ ] A safe, repeatable process for making fast changes

Start small. A one-page personal site is the ideal first project. You can add databases, login, payments, and other complexity later.

Prepared September 2026

1. Set up the foundation

Do these once. Keep the accounts personal, enable two-factor authentication, and save recovery codes in a password manager.

1. Create the core accounts
   Create a GitHub account, a Google account for Firebase, and an OpenAI account with access to ChatGPT and Codex.

2. Install the essential tools
   Install Git, Node.js LTS, a modern browser, and Codex. An editor such as VS Code is useful, even if Codex does most of the editing.

3. Tell Git who you are
   Open Terminal (macOS) or PowerShell (Windows) and configure the name and email that will appear in your commits.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4. Connect your machine to GitHub
   The simplest path is GitHub Desktop. If you prefer the command line, generate an SSH key, add it to the SSH agent, and add the public key to GitHub. Never share the private key.

```
ssh-keygen -t ed25519 -C "you@example.com"
```

5. Create a workspace folder
   Keep projects in one easy-to-find place, such as `Projects` or `code`. Each website gets its own folder and GitHub repository.

Quick check
In a new terminal, run `git --version`, `node --version`, and `npm --version`. Each should print a version number.

If command-line setup feels awkward, ask Codex to inspect the installed tools and explain exactly what is missing. Do not paste passwords, recovery codes, private SSH keys, or secret API keys into a chat.

2. Build the first version

Use a simple stack for the first site: React + TypeScript + Vite is fast, common, and easy to host as static files.

6 Create the project

In your workspace folder, scaffold the site. Choose a short lowercase project name.

```
npm create vite@latest my-site -- --template react-ts
cd my-site
npm install
npm run dev
```

7 Open the folder in Codex

Give Codex access only to this project folder. Start with a clear request: what the site is for, who it serves, the visual mood, and the one action visitors should take.

A good first prompt

Build a polished one-page website for [purpose]. Use React and TypeScript. Keep it responsive and accessible. First inspect the project, then propose a short plan. After implementation, run lint and build. Do not deploy.

8 Work in small passes

Ask for one coherent change at a time: page structure, typography, colors, a section, mobile layout, then polish. Review each pass in the browser before requesting the next.

9 Use ChatGPT as the thinking partner

Use ChatGPT for audience, positioning, copy, page structure, names, and alternatives. Use Codex for changes that need to read files, edit code, run the site, and verify results.

10 Give Codex durable project rules

Add an `AGENTS.md` file with the stack, commands, style principles, important boundaries, and the rule that production deployment requires explicit approval.

Keep control

Before accepting a change, ask: What files changed? What did you verify? Are there any assumptions or remaining risks? Then look at the actual site yourself.

3. Save it, preview it, publish it

11 Create the GitHub repository

Create an empty repository on GitHub. Make it private unless you intentionally want the code public. Never commit `.env` files, service-account files, private keys, or credentials.

12 Make the first commit

Version history is your safety net. Commit a working state before major experiments.

```
git init
git add .
git commit -m "Create initial website"
git branch -M main
git remote add origin git@github.com:YOUR-NAME/my-site.git
git push -u origin main
```

13 Create a Firebase project

In the Firebase console, create a new project with a unique project ID. Start with Hosting only. Turn on billing only when a feature actually requires it, and set budget alerts if billing is enabled.

14 Install and connect Firebase

The Firebase CLI currently requires Node.js 18 or later. Log in, then initialize Hosting from the website folder. For a Vite site, the public directory is usually `dist`; configure it as a single-page app if you use client-side routing.

```
npm install -g firebase-tools
firebase login
firebase init hosting
```

15 Build and test

Run the checks before every publish. Fix errors rather than ignoring them.

```
npm run lint
npm run build
firebase emulators:start
```

16 Publish a preview first

Use a temporary preview channel so you can check the production build on a phone and share it for feedback without changing the live site.

```
firebase hosting:channel:deploy preview
```

17 Deploy live deliberately

Only after reviewing the preview, publish Hosting. Confirm the active Firebase project first if you manage more than one.

```
firebase use
firebase deploy --only hosting
```

4. Adopt the fast iteration loop

Speed comes from short, reversible cycles - not from making a giant request and hoping it works.

1	**Choose**	Pick one visible outcome.
2	**Describe**	Give context, constraints, and acceptance criteria.
3	**Implement**	Let Codex inspect and make the scoped change.
4	**Verify**	Run lint/build/tests and review desktop + mobile.
5	**Commit**	Save the known-good state with a clear message.
6	**Preview**	Share a Firebase preview when feedback is useful.
7	**Ship**	Deploy live only when you intentionally approve it.

NON-NEGOTIABLE HABITS

[ ] Read the diff before committing. A green build does not prove the design is right.

[ ] Keep secrets out of Git. Put local secrets in .env files that are ignored by Git.

[ ] Ask before destructive actions, dependency upgrades, database migrations, or live deployments.

[ ] Use branches for risky experiments; keep main deployable.

[ ] Test the real experience: links, forms, keyboard navigation, phone layout, and page speed.

[ ] Commit frequently enough that you can comfortably undo a bad idea.

When something breaks
Stop adding features. Copy the exact error, say what you expected, and ask Codex to diagnose before fixing. After the fix, rerun the failed check and the full build.

A sensible next step

After the first static site is comfortable, learn branches and pull requests, connect a custom domain, add automated GitHub preview deployments, and only then explore Firebase Authentication, Firestore, Functions, analytics, or payments.

First-week checklist

Aim for one small, finished site - not a grand platform.

DAY 1	Create accounts; install Git, Node.js, Codex, and an editor; confirm the commands work.
DAY 2	Create a one-page Vite site; write the first useful copy; view it locally.
DAY 3	Use Codex for layout and responsive styling; review every change.
DAY 4	Create the GitHub repository; commit and push a clean working version.
DAY 5	Create the Firebase project; configure Hosting; publish a preview.
DAY 6	Test on phone and desktop; fix accessibility, copy, and broken links.
DAY 7	Deploy live, share it with three people, and collect specific feedback.

Definition of done
The site loads from its Firebase URL, looks good on a phone, has no obvious broken links, passes lint and build, contains no secrets, and the live version matches a committed Git revision.

OFFICIAL REFERENCES

Git installation - https://git-scm.com/install/

GitHub SSH key setup - https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

Node.js downloads - https://nodejs.org/en/download

Firebase Hosting quickstart - https://firebase.google.com/docs/hosting/quickstart

Firebase local testing and preview channels - https://firebase.google.com/docs/hosting/test-preview-deploy

OpenAI Codex resources - https://developers.openai.com/codex/

Commands and product details can change. Use the linked official documentation if a screen or command differs from this guide.